

Daftar Fungsi Python untuk Random Variable

Armein Z. R. Langi

Pendahuluan

Dokumen ini merangkum fungsi-fungsi Python yang umum digunakan untuk bekerja dengan **random variable diskrit, kontinu, dan bivariate**, beserta contoh penggunaannya. Fokus utama adalah pada pustaka:

- `random` untuk simulasi sederhana,
- `numpy` untuk komputasi numerik dan pembangkitan sampel,
- `scipy.stats` untuk distribusi probabilitas,
- `matplotlib` untuk visualisasi.

Pustaka yang digunakan

```
import random
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

Random Variable Diskrit

Random variable diskrit memiliki nilai yang dapat dihitung satu per satu, misalnya jumlah sukses, hasil lemparan dadu, atau jumlah kedatangan kejadian.

Bilangan acak diskrit sederhana

`random.randint(a, b)`

Menghasilkan bilangan bulat acak dari `a` sampai `b`, inklusif.

```
import random
x = random.randint(1, 6)
print(x)
```

Contoh penggunaan: simulasi satu lemparan dadu.

`random.choice(seq)`

Memilih satu elemen acak dari suatu daftar.

```
warna = ["merah", "hijau", "biru"]
pilih = random.choice(warna)
print(pilih)
```

numpy.random.randint(low, high, size)

Menghasilkan array bilangan bulat acak. Perhatikan bahwa **high** tidak termasuk.

```
import numpy as np
sampel_dadu = np.random.randint(1, 7, size=10)
print(sampel_dadu)
```

Bernoulli

Random variable Bernoulli bernilai 1 untuk sukses dan 0 untuk gagal.

Simulasi dengan random.random()

```
p = 0.3
x = 1 if random.random() < p else 0
print(x)
```

numpy.random.binomial(n, p, size) untuk Bernoulli

Karena Bernoulli adalah Binomial dengan $n=1$:

```
sampel_bernoulli = np.random.binomial(1, 0.3, size=20)
print(sampel_bernoulli)
```

Binomial

numpy.random.binomial(n, p, size)

Menghasilkan sampel dari distribusi Binomial.

```
sampel_binomial = np.random.binomial(n=10, p=0.4, size=1000)
print(sampel_binomial[:10])
```

scipy.stats.binom.pmf(k, n, p)

Menghitung peluang tepat k sukses.

```
from scipy import stats
peluang = stats.binom.pmf(k=3, n=10, p=0.4)
print(peluang)
```

scipy.stats.binom.cdf(k, n, p)

Menghitung peluang kumulatif $P(X \leq k)$.

```
peluang_kumulatif = stats.binom.cdf(k=3, n=10, p=0.4)
print(peluang_kumulatif)
```

Poisson

numpy.random.poisson(lam, size)

Menghasilkan sampel Poisson.

```
sampel_poisson = np.random.poisson(lam=2.5, size=1000)
print(sampel_poisson[:10])
```

scipy.stats.poisson.pmf(k, mu)

Menghitung peluang tepat k kejadian.

```
peluang = stats.poisson.pmf(k=4, mu=2.5)
print(peluang)
```

scipy.stats.poisson.cdf(k, mu)

Menghitung peluang kumulatif.

```
peluang_kumulatif = stats.poisson.cdf(k=4, mu=2.5)
print(peluang_kumulatif)
```

Geometrik

numpy.random.geometric(p, size)

Menghasilkan sampel dari distribusi Geometrik.

```
sampel_geom = np.random.geometric(p=0.2, size=20)
print(sampel_geom)
```

scipy.stats.geom.pmf(k, p)

Menghitung peluang sukses pertama terjadi pada percobaan ke-k.

```
peluang = stats.geom.pmf(k=4, p=0.2)
print(peluang)
```

Nilai harapan dan variansi diskrit

Jika tersedia array data diskrit:

```
x = np.array([0, 1, 2, 3, 4])
p = np.array([0.1, 0.2, 0.4, 0.2, 0.1])
```

Mean dengan `numpy.sum()`

```
ekspektasi = np.sum(x * p)
print(ekspektasi)
```

Variansi dengan rumus $E[X^2] - (E[X])^2$

```
variansi = np.sum((x**2) * p) - ekspektasi**2
print(variansi)
```

Random Variable Kontinu

Random variable kontinu dapat mengambil nilai pada suatu interval kontinu, misalnya tinggi badan, berat badan, atau waktu tunggu.

Uniform kontinu

`random.random()`

Menghasilkan bilangan acak uniform pada interval $[0, 1)$.

```
u = random.random()
print(u)
```

`random.uniform(a, b)`

Menghasilkan bilangan acak uniform pada interval $[a, b]$.

```
x = random.uniform(10, 20)
print(x)
```

`numpy.random.uniform(low, high, size)`

```
sampel_uniform = np.random.uniform(10, 20, size=1000)
print(sampel_uniform[:10])
```

Normal

`numpy.random.normal(loc, scale, size)`

Menghasilkan sampel dari distribusi Normal.

```
sampel_normal = np.random.normal(loc=170, scale=8, size=1000)
print(sampel_normal[:10])
```

scipy.stats.norm.pdf(x, loc, scale)

Menghitung nilai fungsi densitas peluang (PDF).

```
pdf_175 = stats.norm.pdf(175, loc=170, scale=8)
print(pdf_175)
```

scipy.stats.norm.cdf(x, loc, scale)

Menghitung peluang kumulatif $P(X \leq x)$.

```
cdf_175 = stats.norm.cdf(175, loc=170, scale=8)
print(cdf_175)
```

scipy.stats.norm.ppf(q, loc, scale)

Menghitung kuantil atau inverse CDF.

```
x95 = stats.norm.ppf(0.95, loc=170, scale=8)
print(x95)
```

Ekspensial

numpy.random.exponential(scale, size)

Untuk laju λ , parameter $\text{scale} = 1/\lambda$.

```
sampel_exp = np.random.exponential(scale=1/0.5, size=1000)
print(sampel_exp[:10])
```

scipy.stats.expon.pdf(x, scale)

```
pdf_2 = stats.expon.pdf(2, scale=1/0.5)
print(pdf_2)
```

scipy.stats.expon.cdf(x, scale)

```
cdf_2 = stats.expon.cdf(2, scale=1/0.5)
print(cdf_2)
```

Statistik sampel kontinu

Jika data tersimpan dalam array x:

```
x = np.random.normal(170, 8, size=1000)
```

Mean dengan `numpy.mean()`

```
mean_x = np.mean(x)
print(mean_x)
```

Variansi dengan `numpy.var()`

```
var_pop = np.var(x)          # variansi populasi
var_sampel = np.var(x, ddof=1) # variansi sampel
print(var_pop, var_sampel)
```

Simpangan baku dengan `numpy.std()`

```
std_pop = np.std(x)
std_sampel = np.std(x, ddof=1)
print(std_pop, std_sampel)
```

Random Variable Bivariate

Bivariate berarti ada dua random variable yang diamati bersama, misalnya tinggi dan berat, atau storage dan compute cost.

Membuat sampel bivariate independen

```
x = np.random.normal(170, 8, size=1000)
y = np.random.normal(65, 10, size=1000)
```

Membuat sampel bivariate berkorelasi

`numpy.random.multivariate_normal(mean, cov, size)`

Fungsi ini sangat penting untuk simulasi bivariate normal.

```
mean = [50, 100]
cov = [[25, 14],
       [14, 36]]

sampel = np.random.multivariate_normal(mean, cov, size=1000)
x = sampel[:, 0]
y = sampel[:, 1]
```

Kovariansi

numpy.cov(m, rowvar=False)

```
data = np.column_stack((x, y))
matriks_cov = np.cov(data, rowvar=False)
print(matriks_cov)
```

Atau langsung:

```
matriks_cov = np.cov(x, y)
print(matriks_cov)
```

Korelasi

numpy.corrcoef(x, y)

```
matriks_corr = np.corrcoef(x, y)
print(matriks_corr)
```

Koefisien korelasi Pearson terdapat pada elemen di luar diagonal.

scipy.stats.pearsonr(x, y)

Menghasilkan koefisien korelasi sekaligus p-value.

```
r, p_value = stats.pearsonr(x, y)
print(r, p_value)
```

Regresi linear sederhana

scipy.stats.linregress(x, y)

```
hasil = stats.linregress(x, y)
print("slope =", hasil.slope)
print("intercept =", hasil.intercept)
print("rvalue =", hasil.rvalue)
```

Joint distribution empiris untuk data diskrit bivariate

Jika x dan y adalah data diskrit:

```
x = np.random.randint(1, 4, size=1000)
y = np.random.randint(1, 5, size=1000)
```

Menghitung frekuensi gabungan dengan `pandas.crosstab()`

```
import pandas as pd

tabel = pd.crosstab(x, y)
print(tabel)
```

Mengubah menjadi peluang gabungan empiris

```
joint_prob = pd.crosstab(x, y, normalize=True)
print(joint_prob)
```

Mean, variansi, dan kovariansi bivariate

```
mean_x = np.mean(x)
mean_y = np.mean(y)
var_x = np.var(x, ddof=1)
var_y = np.var(y, ddof=1)
cov_xy = np.cov(x, y, ddof=1)[0, 1]

print(mean_x, mean_y, var_x, var_y, cov_xy)
```

Visualisasi

Visualisasi sangat penting untuk memahami random variable.

Histogram untuk random variable tunggal

```
x = np.random.normal(170, 8, size=1000)
plt.hist(x, bins=30, edgecolor='black')
plt.xlabel("Nilai")
plt.ylabel("Frekuensi")
plt.title("Histogram Sampel Normal")
plt.show()
```

Scatter plot untuk random variable bivariate

```
mean = [50, 100]
cov = [[25, 14], [14, 36]]
sampel = np.random.multivariate_normal(mean, cov, size=500)

plt.scatter(sampel[:, 0], sampel[:, 1], alpha=0.5)
plt.xlabel("X")
plt.ylabel("Y")
```



```
plt.title("Scatter Plot Data Bivariate")
plt.show()
```

Overlay histogram dan PDF teoritis

```
x = np.random.normal(170, 8, size=1000)
plt.hist(x, bins=30, density=True, edgecolor='black', alpha=0.7)

xx = np.linspace(140, 200, 400)
pdf = stats.norm.pdf(xx, loc=170, scale=8)
plt.plot(xx, pdf)

plt.xlabel("x")
plt.ylabel("Density")
plt.title("Histogram dan PDF Normal")
plt.show()
```

Ringkasan Fungsi Penting

Random variable diskrit

Kebutuhan	Fungsi Python	Contoh
Bilangan bulat acak	<code>random.randint(a,b)</code>	<code>random.randint(1,6)</code>
Sampel integer banyak	<code>np.random.randint(low, high, size)</code>	<code>np.random.randint(1,7,100)</code>
Bernoulli	<code>np.random.binomial(1,p,size)</code>	<code>np.random.binomial(1,0.3,20)</code>
Binomial sampel	<code>np.random.binomial(n,p,size)</code>	<code>np.random.binomial(10,0.4,1000)</code>
Binomial PMF	<code>stats.binom.pmf(k,n,p)</code>	<code>stats.binom.pmf(3,10,0.4)</code>
Poisson sampel	<code>np.random.poisson(lam,size)</code>	<code>np.random.poisson(2.5,1000)</code>
Poisson PMF	<code>stats.poisson.pmf(k,mu)</code>	<code>stats.poisson.pmf(4,2.5)</code>
Geometrik sampel	<code>np.random.geometric(p,size)</code>	<code>np.random.geometric(0.2,20)</code>
Mean diskrit	<code>np.sum(x*p)</code>	<code>np.sum(x*p)</code>
Variansi diskrit	<code>np.sum((x**2)*p) - E**2</code>	sesuai rumus

Random variable kontinu

Kebutuhan	Fungsi Python	Contoh
Uniform [0,1)	<code>random.random()</code>	<code>random.random()</code>
Uniform [a,b]	<code>random.uniform(a,b)</code>	<code>random.uniform(10,20)</code>

Kebutuhan	Fungsi Python	Contoh
Uniform banyak	<code>np.random.uniform(a,b,size)</code>	<code>np.random.uniform(10,20,1000)</code>
Normal sampel	<code>np.random.normal(loc,scale,size)</code>	<code>np.random.normal(170,8,1000)</code>
Normal PDF	<code>stats.norm.pdf(x,loc,scale)</code>	<code>stats.norm.pdf(175,170,8)</code>
Normal CDF	<code>stats.norm.cdf(x,loc,scale)</code>	<code>stats.norm.cdf(175,170,8)</code>
Normal inverse CDF	<code>stats.norm.ppf(q,loc,scale)</code>	<code>stats.norm.ppf(0.95,170,8)</code>
Ekspensial sampel	<code>np.random.exponential(scale,size)</code>	<code>np.random.exponential(2,1000)</code>
Mean sampel	<code>np.mean(x)</code>	<code>np.mean(x)</code>
Variansi sampel	<code>np.var(x,ddof=1)</code>	<code>np.var(x,ddof=1)</code>
Simpangan baku	<code>np.std(x,ddof=1)</code>	<code>np.std(x,ddof=1)</code>

Random variable bivariate

Kebutuhan	Fungsi Python	Contoh
Bivariate normal	<code>np.random.multivariate_normal(mean,cov,size)</code>	<code>np.random.multivariate_normal(mean,cov,size)</code>
Kovariansi	<code>np.cov(x,y)</code>	<code>np.cov(x,y)</code>
Korelasi	<code>np.corrcoef(x,y)</code>	<code>np.corrcoef(x,y)</code>
Korelasi Pearson	<code>stats.pearsonr(x,y)</code>	<code>stats.pearsonr(x,y)</code>
Regresi linear	<code>stats.linregress(x,y)</code>	<code>stats.linregress(x,y)</code>
Tabel joint empiris	<code>pd.crosstab(x,y)</code>	<code>pd.crosstab(x,y)</code>

Contoh Mini Proyek

Simulasi dadu diskrit

```
import numpy as np

x = np.random.randint(1, 7, size=1000)
print("Mean =", np.mean(x))
print("Variansi =", np.var(x, ddof=1))
```

Simulasi tinggi badan kontinu

```
x = np.random.normal(170, 8, size=1000)
print("Mean =", np.mean(x))
print("Std =", np.std(x, ddof=1))
```

Simulasi dua variabel berkorelasi

```
mean = [100, 200]
cov = [[100, 60], [60, 80]]
data = np.random.multivariate_normal(mean, cov, size=1000)

x = data[:, 0]
y = data[:, 1]

print("Covariance matrix:")
print(np.cov(x, y))
print("Correlation matrix:")
print(np.corrcoef(x, y))
```

Penutup

Dalam Python, kombinasi `numpy`, `scipy.stats`, `random`, dan `matplotlib` sudah sangat memadai untuk:

- mensimulasikan random variable,
- menghitung PMF, PDF, dan CDF,
- mencari mean, variansi, kovariansi, dan korelasi,
- membuat visualisasi histogram dan scatter plot,
- mempelajari hubungan antar variabel acak secara empiris maupun teoritis.

Dokumen ini dapat dijadikan referensi awal untuk pembelajaran probabilitas, statistika, simulasi Monte Carlo, dan analisis data berbasis Python.